



Measuring the Tokenization Premium

A Cost Audit for Underserved Language Communities

Avijit Roy

John Jay College of Criminal Justice
The City University of New York

Proma Roy

The City College of New York
The City University of New York

Hrishitva Patel

University of Texas
at San Antonio



Read the paper: arXiv Link

Workshop on Language Models for Underserved Communities

LM4UC @ IJCAI 2026 · 16 August 2026

github.com/HeyAvijitRoy/tea-benchmark

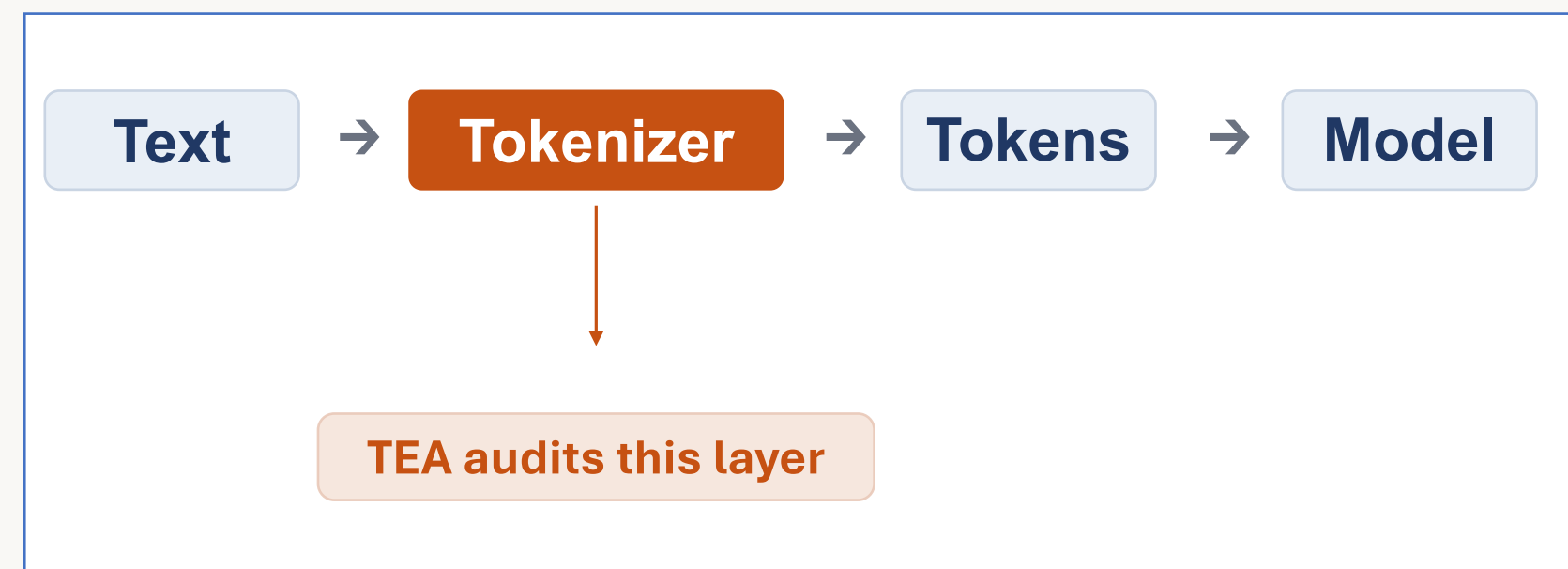
We benchmark the model. We rarely audit the tokenizer underneath it.

MODEL EVALUATION USUALLY ASKS

- 01 Can it understand?**
Language comprehension
- 02 Can it answer well?**
Accuracy · reasoning · usefulness
- 03 Can it preserve meaning?**
Translation · culture · semantics

These questions begin after text has already been encoded.

Before inference, one decision has already happened:



Same meaning does not necessarily enter the model with the same representation cost.

That cost is measurable from the vocabulary file and a parallel corpus before a single output token is generated.

NO MODEL INFERENCE REQUIRED

THE QUESTION WE ASK

How much does equivalent instructional content cost to represent across languages before inference even begins?

- RQ1** How large is the tokenization premium?
- RQ2** Does it change across tokenizer families?
- RQ3** What does it imply for usable context and deployment?

The same error message fragments into 8 tokens in English and 22 in Tamil

Corpus item T1-10 • "IndexError: list index out of range" • GPT-4o o200k_base

English	Index Error : list index out of range	8 tokens
Bengali	Index Error : তাল িকার সূ চ ক সীম ার বাইরে	11 tokens
Hindi	इ ंडे क्स त्रु टि : सूची इंड ेक्स सीमा बाहर	11 tokens
Tamil	க ுற ிய ீ ட்டு ப ிழ ை : பட்ட ிய லில் குற ிய ீ டு வர ம்ப ிற்சு வெளிய ே உள்ளது	22 tokens
Yoruba	À ş ì şe Index : à k ój ọ atọ ka kò sí ní ì wọ ` n	19 tokens

No model has run at this point. The difference is produced entirely by the encoder vocabulary.

Extra tokens become cost, lost context, and latency before inference begins

01

API billing

Commercial services price by token. Identical instructional content carries a different price depending on the language it was written in, with no change in what the learner receives.

02

Usable context

A nominal window is counted in tokens, not in meaning. A longer encoding fills it faster, so less conversation history, documentation, and fewer worked examples survive in one tutoring session.

03

Local deployment

Open weights remove the bill but not the sequence. The overhead returns as memory pressure and inference latency on exactly the hardware low-connectivity settings can afford.

Tokenizer design changes what a multilingual system can carry, compute, and afford.

The TEA Benchmark isolates the tokenizer from model inference

Why a custom programming corpus: realistic beginner debugging support, retained code identifiers, and controlled parallel content.

Purpose: Introductory Python education is a high-impact use case. We designed a 120-item parallel corpus of beginner debugging prompts.

Retained code identifiers: Keywords like `TypeError`, `len()`, `NoneType`, `IndexError`, and variable names stay in English. That reflects actual code-switching classroom practice, not synthetic literal translation.

CORPUS STRUCTURE

- 35 Tier 1:** short technical phrases, error names, diagnostics
- 50 Tier 2:** one- to three-sentence explanations of a bug or a fix
- 35 Tier 3:** longer tutoring passages with concepts and code examples

TOKENIZERS AUDITED

GPT-4o o200k_base

Qwen2.5-7B

Mistral-7B-v0.1

VALIDATION SCOPE

Bengali and Hindi: reviewed by speakers with programming experience. The primary validated cases.

Arabic, Tamil, Yoruba: machine translated and inspected. Exploratory comparisons only.

No model inference is used. Token counts are computed from vocabulary files over parallel corpus items.

The evaluation framework quantifies representation overhead

Two durable ratios and one illustrative cost translation.

TFR

Token Fertility Ratio

$$\text{TFR} = \text{tokens}(\text{target}) / \text{tokens}(\text{English})$$

Calculated per parallel item under the same tokenizer. A ratio of 1.0 represents parity with English. A value of 1.56 means the target language requires 56% more tokens than English for the same exact meaning.

ECW

Effective Context Window

$$\text{ECW} = \text{Nominal Window} / \text{TFR}$$

Quantifies the real capacity penalty. Model attention limits are counted in raw tokens, not meaning. Thus, a higher TFR directly shrinks the effective semantic workspace available for chat history and document retrieval.

P

Illustrative API Premium

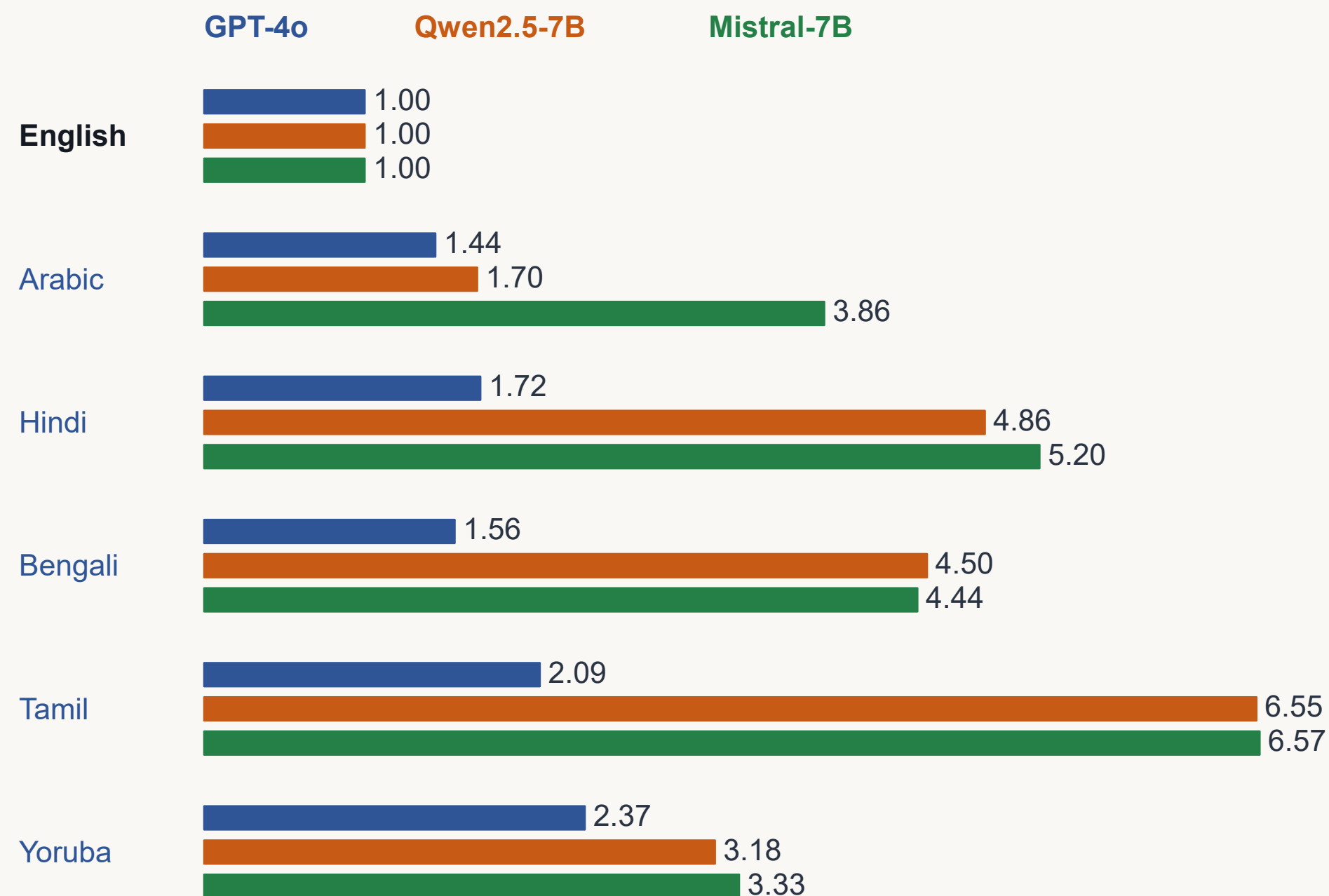
$$\text{Premium} = \text{Price} \times (\text{TFR} - 1) \times \text{mean_English_tokens} \times 1000$$

Translates structural token fertility differences into financial pricing. Using commercial GPT-4o input rate cards (\$2.50 per million input tokens), this captures the systematically higher bill paid per 1,000 queries.

Cost numbers in the talk use GPT-4o input pricing from May 2026 for illustration; the ratios are the stable research result.

Every tested non-English language pays a premium under every tokenizer

Mean token fertility ratio across 120 items · English baseline = 1.00



Source: TEA paper, Table 1.

Key findings

1.56× vs ≈4.5×

Bengali under GPT-4o versus Qwen2.5; Mistral is 4.44×. The comparison shows that tokenizer design materially changes sequence overhead.

6.55× / 6.57×

Tamil shows the highest open-weight penalty in the corpus.

The deployment decision is not only cloud versus local; it is also sequence overhead versus billing cost.

Script family alone does not explain the tokenization premium

Latin-script Yoruba is more expensive than Bengali under GPT-4o, so script is not enough.

Yoruba

2.37x

Highest GPT-4o premium in the corpus, written in Latin script. Tonal marks and underdots [Àşìşe] – produce sequences the vocabulary covers poorly.

Bengali

1.56x

Lower GPT-4o premium, written in a Brahmic alphasyllabary with conjuncts and diacritics. The script that looks harder to segment is the cheaper one here.

Vocabulary coverage, diacritic handling, and how often a language appeared in tokenizer training all matter alongside script family. Fairness has to be measured per language instead of inferred from the alphabet.

Retained English terms suppress the Bengali penalty rather than inflate it

Bengali token fertility under GPT-4o, all 120 items against the clean-Bengali subset of 62.

The code-switching suppression effect

Educational reality

Programming instruction naturally keeps English technical terms such as `TypeError`, `NoneType`, and `len` alongside native explanations.

Methodological check

We classified Bengali items by script share. Clean Bengali means at least 75% meaningful Bengali-script characters.

Clear result

Clean Bengali is more expensive in every tier. English identifiers pull the measured premium down, so 1.56× is conservative.

Bengali TFR Sensitivity (GPT-4o)

Bengali subset	Tier 1 (Phrases)	Tier 2 (Exps)	Tier 3 (Tutors)
All items (n=120)	1.65	1.55	1.50
Clean only (n=62)	2.11	1.79	1.79

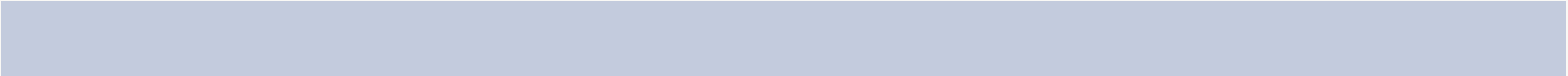
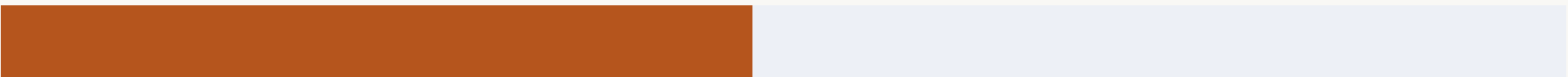
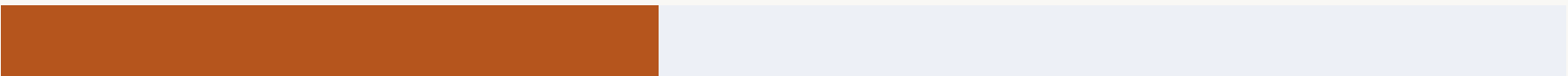
Clean = at least 75% Bengali-script characters

Mixed = 40–75%

English-retained = below 40%.

A nominal 128k window carries only ~82k English-equivalent content in Bengali

English-equivalent content that fits inside a nominal 128,000-token window under GPT-4o.

	Usable share of the window	Effective context	Cost / 1k calls
English		128,000 - 100%	baseline
Arabic		89,148 - 70%	+\$0.06
Bengali		81,967 - 64%	+\$0.07
Hindi		74,461 - 58%	+\$0.09
Tamil		61,121 - 48%	+\$0.14
Yoruba		53,951 - 42%	+\$0.18

64%

of the nominal window remains usable in Bengali — about 82,000 English-equivalent tokens of content.

42%

remains in Yoruba. That is the effective room for prior turns, examples, and retrieved documents.

Estimates use a 128,000-token nominal window and GPT-4o input pricing of \$2.50 per million tokens, May 2026. The ratios are the durable result.

The premium is paid again when multilingual data is built

A 100,000-token English instructional corpus, translated or curated in other languages using GPT-4o ratios from Table 1.

English source

100k

tokens of curated
instructional content
baseline



Bengali

156k

+56% tokens for the same
content



Tamil

209k

+109% tokens for the same
content



Yoruba

237k

+137% tokens for the same
content

Storage, transmission bandwidth, annotation throughput and curation time all scale with that ratio, a structural barrier to resource development that operates independently of model capability.

Token efficiency is not answer quality

Four boundaries on what this audit supports.

Corpus scale

120 items in one technical-education domain. A workshop-scale audit, not a general ranking of tokenizers across languages.

Sensitivity coverage

The code-switching analysis covers Bengali only. All five target languages retain English identifiers, so the suppression effect may extend.

Validation depth

Bengali and Hindi are human-validated. Arabic, Tamil and Yoruba are exploratory and should be read as motivation for wider work.

Pricing

Dollar figures are illustrative and tied to May 2026 rates. The ratios are what should be carried forward.

**A tokenizer that produces fewer tokens can still sit underneath a weaker model.
TEA measures infrastructure cost, and nothing beyond that.**

Four reporting changes would make tokenization cost visible

These are reporting practices existing model cards and benchmark papers could adopt now.

01 Report tokenizer efficiency by language

Model cards and benchmark papers should carry language-level tokenization statistics, particularly for the languages least represented in the vocabulary.

02 Publish context-equivalent capacity

A 128k claim means different things in different languages. The adjustment is one division, and it belongs next to the headline number.

03 Audit terminology-heavy domains separately

Programming, legal assistance, and health guidance behave differently from general text because of code-switching and domain vocabulary.

04 Measure sequence overhead before choosing deployment

The API-versus-local decision for a low-connectivity classroom depends on numbers that are currently not published anywhere.

None of the four require new research. They require making an invisible denominator visible.

Tokenization is a measurable equity layer beneath the model

1.56×

Bengali GPT-4o token premium across the 120-item corpus, and a conservative estimate.

4.50×

Bengali under open-weight tokenizers — the cost of avoiding the API bill.

82k

English-equivalent content that fits a nominal 128k window in Bengali.

Equal access to a model does not guarantee equal treatment by the infrastructure underneath it. Tokenization is one layer of that infrastructure, it is measurable, and the measurement needs no GPU and no inference.

A vocabulary file and a parallel corpus are enough.



Thank you

Questions and discussion welcome.

CONTACT

Avijit Roy | aroy@jjay.cuny.edu

Proma Roy | proma.roy33@stu-mail.ccny.cuny.edu

Hrishitva Patel | hrishitva.patel@utsa.edu

github.com/HeyAvijitRoy/tea-benchmark

ACKNOWLEDGMENT

Supported in part by the NSF ACCESS program, allocation CIS260616, “Dataset Development and Fine-Tuning of Language Models for Low-Resource Bengali Programming Assistance.”
Analysis was conducted on Indiana University Jetstream2.